



Ingeniería en Software

Timeout de inactividad

Seguridad de la información

Francisco Alejandro Zúñiga Barajas - 233435

Rodrigo Vazquez Reyes - 233425

Tuxtla Gutierrez, 2026

1. Introducción

En el desarrollo de aplicaciones móviles modernas, la seguridad representa uno de los aspectos más importantes, especialmente cuando se trabaja con aplicaciones que manejan sesiones de usuario, autenticación y datos sensibles. Flutter permite crear aplicaciones multiplataforma utilizando Dart, mientras que Android proporciona herramientas nativas que permiten reforzar la seguridad del sistema mediante Kotlin.

Durante esta práctica se desarrolló una aplicación Flutter enfocada en la implementación de mecanismos de seguridad relacionados con la gestión de sesiones e inactividad del usuario. El objetivo principal consistió en implementar un sistema de timeout automático capaz de cerrar la sesión después de cierto tiempo sin interacción, además de integrar medidas de seguridad nativas del sistema Android.

La aplicación desarrollada incorpora autenticación básica, almacenamiento seguro de tokens mediante almacenamiento cifrado, detección de actividad del usuario, cierre automático por inactividad y validaciones de seguridad relacionadas con ubicación simulada (Fake GPS). Asimismo, se integró comunicación entre Flutter y código nativo Kotlin utilizando MethodChannels.

El desarrollo permitió comprender la interacción entre Flutter y Android nativo, así como la importancia de aplicar principios de seguridad desde la arquitectura de la aplicación.

2. Recursos Utilizados

Hardware

- Laptop con Fedora Linux 41
- Dispositivo físico Motorola G55 5G

Software

- Flutter SDK 3.44.1
- Dart SDK
- Visual Studio Code
- Android SDK
- Kotlin

- Gradle
- ADB (Android Debug Bridge)

Librerías Utilizadas

- flutter_secure_storage
- permission_handler
- safe_device

Tecnologías Implementadas

- Flutter
- Dart
- Kotlin
- Android MethodChannels
- Secure Storage
- Timers y gestión de estados
- Manejo de permisos Android

3. Desarrollo (Timeout)

El sistema de timeout implementado tiene como propósito cerrar automáticamente la sesión del usuario después de un periodo determinado de inactividad. Esto reduce el riesgo de accesos no autorizados cuando el dispositivo permanece desbloqueado.

Para lograrlo, se implementó un componente llamado `InactivityManager`, encargado de monitorear constantemente la interacción del usuario mediante eventos táctiles.

El flujo general funciona de la siguiente manera:

1. El usuario inicia sesión.
2. Se crea un temporizador interno.
3. Cada interacción táctil reinicia el temporizador.
4. Si el tiempo expira sin actividad, se ejecuta automáticamente el logout.
5. El token almacenado es eliminado del almacenamiento seguro.

El sistema utiliza la clase `Timer` de Dart junto con `Listener` para detectar eventos de interacción del usuario.

También se implementó un contador visual que muestra el tiempo restante antes del cierre automático de sesión. Este contador se actualiza dinámicamente cada segundo.

Adicionalmente, el tiempo de inactividad puede configurarse desde la propia interfaz de usuario, permitiendo modificar el tiempo límite según las necesidades de la aplicación.

Características implementadas

- Reinicio automático del temporizador al detectar interacción.
 - Cierre de sesión automático.
 - Persistencia segura del token.
 - Configuración dinámica del timeout.
 - Contador visual de tiempo restante.
 - Detección de aplicación enviada a segundo plano.
 - Restauración automática del control al volver a foreground.
-

3.1. Implementación de la Seguridad Nativa (Kotlin)

Además del timeout implementado en Flutter, se integraron mecanismos de seguridad nativos mediante Kotlin.

La aplicación utiliza `MethodChannel` para comunicar Flutter con Android nativo. Esto permitió ejecutar funciones específicas del sistema operativo que no están disponibles directamente desde Flutter.

Entre las funcionalidades implementadas destacan:

Protección FLAG_SECURE

Se utilizó `FLAG_SECURE` para evitar:

- Capturas de pantalla.
- Grabación de pantalla.
- Visualización del contenido en aplicaciones recientes.

Esta protección se activa automáticamente al ingresar al login y se desactiva al salir de la pantalla.

Detección de Fake GPS

La aplicación verifica si el dispositivo utiliza ubicaciones simuladas utilizando:

- La librería `safe_device`.
- Métodos nativos Kotlin.

Si se detecta una ubicación falsa, la aplicación bloquea completamente el acceso mostrando una pantalla de seguridad.

Manejo de permisos

Se implementó validación dinámica de permisos de ubicación utilizando `permission_handler`. Si el usuario niega el permiso, la aplicación restringe el acceso hasta que se otorgue autorización.

Almacenamiento Seguro

El token de sesión se almacena utilizando `flutter_secure_storage`, el cual cifra automáticamente la información utilizando capacidades seguras del sistema Android.

4. Resultados y Conclusiones

La práctica permitió implementar exitosamente un sistema de seguridad funcional utilizando Flutter y Kotlin de manera híbrida. Se logró integrar tanto componentes multiplataforma como herramientas nativas del sistema Android.

El sistema de timeout demostró funcionar correctamente al detectar periodos de inactividad y cerrar automáticamente la sesión del usuario. Asimismo, el contador visual permitió mejorar la experiencia del usuario mostrando el tiempo restante antes del cierre automático.

La integración de Kotlin mediante MethodChannels permitió extender significativamente las capacidades de seguridad de Flutter, especialmente en aspectos relacionados con protección de pantalla y detección de ubicaciones falsas.

Durante el desarrollo también se comprendió la importancia de la correcta gestión de estados, temporizadores y almacenamiento seguro dentro de aplicaciones móviles.

El uso de Flutter facilitó el desarrollo rápido de interfaces modernas y reactivas, mientras que Dart permitió mantener una arquitectura organizada gracias a su tipado fuerte y orientación a objetos.

Adicionalmente, trabajar desde Linux y utilizar VS Code junto con la terminal Bash simplificó considerablemente la ejecución, depuración y despliegue de la aplicación hacia el dispositivo físico Android.

En conclusión, la práctica permitió reforzar conocimientos sobre:

- Seguridad móvil.
 - Integración Flutter + Android nativo.
 - Gestión de sesiones.
 - Persistencia segura.
 - Timers y eventos.
 - Comunicación entre plataformas.
 - Manejo de permisos Android.
-

5. Anexos

5.1 Evidencias



5.2. Errores Presentados Durante la Práctica

Error 1 – NDK Corrupto

Durante la compilación inicial se presentó un problema relacionado con el Android NDK.

Error:

- `NDK did not have a source.properties file`

Causa:

- Descarga corrupta del NDK.

Solución:

- Eliminación manual del directorio corrupto.
 - Reinstalación automática mediante SDK Manager.
-

Error 2 – Parámetro requerido no proporcionado

Error:

- `Required named parameter 'timeoutMinutes' must be provided`

Causa:

- El constructor de `HomeScreen` requería un parámetro obligatorio que no estaba siendo enviado.

Solución:

- Agregar correctamente el parámetro al constructor.
-

Error 3 – Conflicto entre int y double

Error:

- Problemas de tipado entre `double`, `int` y `Duration`.

Causa:

- Se intentó representar segundos utilizando variables diseñadas originalmente para minutos.

Solución:

- Refactorización completa para trabajar únicamente con segundos enteros.
-

5.3. Documentación de Prompts Utilizados

Durante el desarrollo se utilizaron herramientas de inteligencia artificial para resolver problemas técnicos específicos relacionados con Flutter y Android.

Ejemplos de prompts utilizados:

- “¿Cómo implementar un sistema de timeout por inactividad en Flutter?”
- “¿Cómo detectar interacción del usuario utilizando Listener en Flutter?”
- “¿Cómo cerrar sesión automáticamente después de cierto tiempo?”
- “¿Cómo utilizar flutter_secure_storage?”
- “¿Cómo implementar FLAG_SECURE usando Kotlin?”
- “¿Cómo detectar Fake GPS en Android?”
- “¿Cómo comunicar Flutter con Kotlin mediante MethodChannels?”
- “¿Cómo solucionar errores de NDK corrupto en Flutter?”
- “¿Cómo ejecutar una aplicación Flutter en un dispositivo físico Android desde Linux?”

La asistencia proporcionada permitió acelerar significativamente la resolución de errores y comprender mejor conceptos relacionados con seguridad móvil e integración nativa.